Visualizador de un UR3e por TCP/IP

Memoria de la práctica · código en github.com/godzilin/Practica1UR

Cuatro programas que leen por la red el estado de un brazo **UR3e** (real o URSim) y lo muestran en 3D con **PyBullet**, de menos a más: una interfaz, otra más rápida, varios robots y, por último, una aplicación que además da órdenes al robot. Todos se lanzan con F5 y leen las IP de `robots.json`.

A · Interfaz primaria (primaria.py)

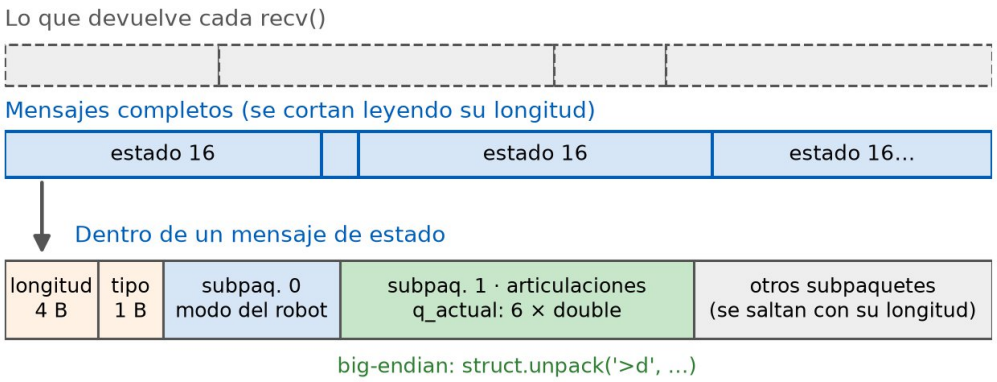


Fig. 1 · De los bytes que llegan por TCP a los 6 ángulos del robot.

- **Un recv() no es un mensaje:** en un minuto hubo 1110 `recv()` de 11 tamaños distintos. Se acumula en un buffer y se corta leyendo la longitud de cada mensaje.
- Del mensaje de estado (tipo 16) solo interesa el **subpaquete 1**; los demás se saltan con su longitud.
- Números en **big-endian** (>). Si se corta la conexión, **reconecta solo**.

B · Interfaz de tiempo real (tiempo_real.py)

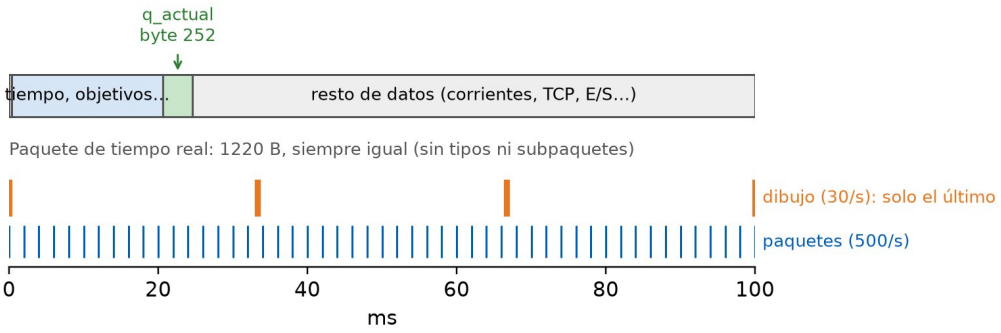


Fig. 2 · Paquete del puerto 30003 y ritmo de llegada frente al de dibujo.

- Estructura fija: los ángulos están **siempre en el byte 252**. No hace falta recorrer subpaquetes.
- Llegan ~500 paquetes/s (URSim da ~480), demasiados para dibujarlos todos: se leen todos, **se guarda solo el último** y se dibuja 30 veces por segundo.

C · Varios robots (varios_robots.py)

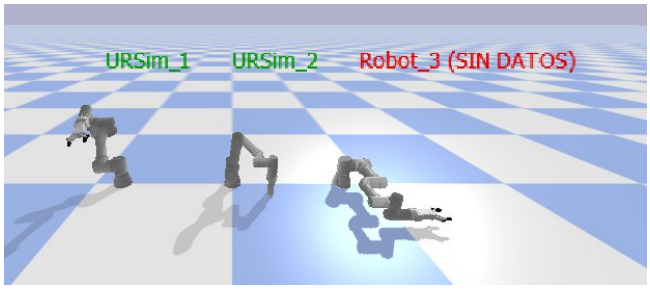


Fig. 3 · Dos URSim y un robot apagado.

- **Un hilo por robot**, cada uno con su socket y su buffer: si uno se cuelga, solo se bloquea su hilo.
- Los hilos dejan el último estado en un diccionario protegido con un **Lock**; un único bucle lo lee y dibuja.
- Si un robot deja de enviar 1 s, su nombre pasa a rojo **«SIN DATOS»** y se queda quieto.

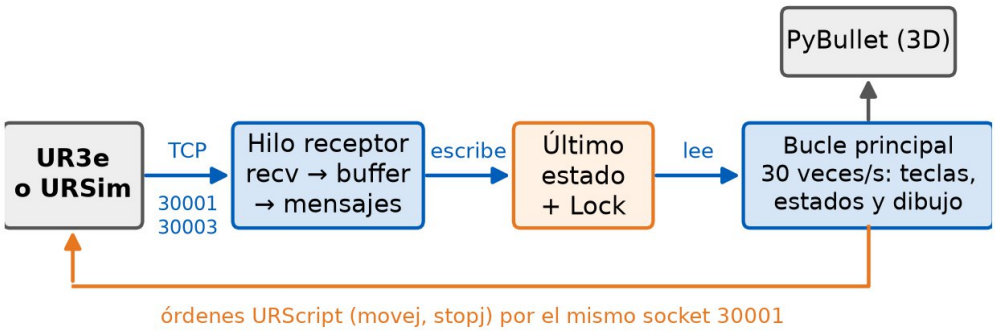


Fig. 4 · Estructura de C y D: recibir, guardar el último estado y dibujar.

D · Enseñar y reproducir (ensenar_reproducir.py)

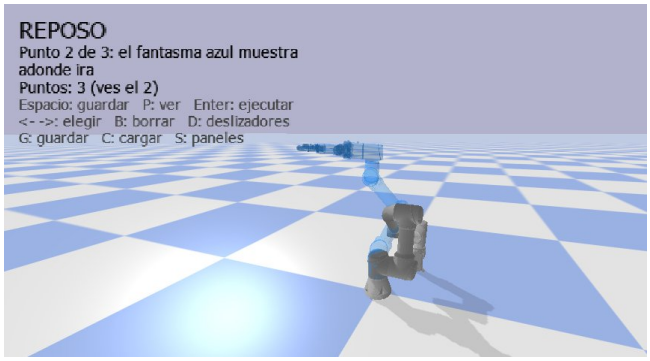


Fig. 5 · Gris: el robot. Azul: el punto que va a ejecutar.

- **Enseñar:** Espacio guarda la postura del robot como punto.
- **Previsualizar:** un robot «fantasma» azul recorre los puntos sin mover el real.
- **Ejecutar:** envía un `movej` por punto por el mismo socket 30001 y **confirma** cada uno.

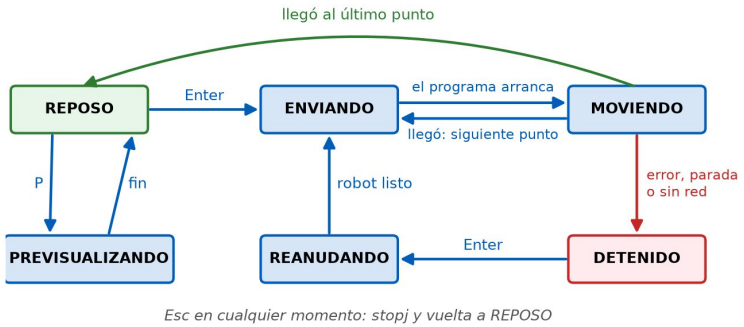


Fig. 6 · Máquina de estados. Ante un error o parada se detiene y muestra el motivo.

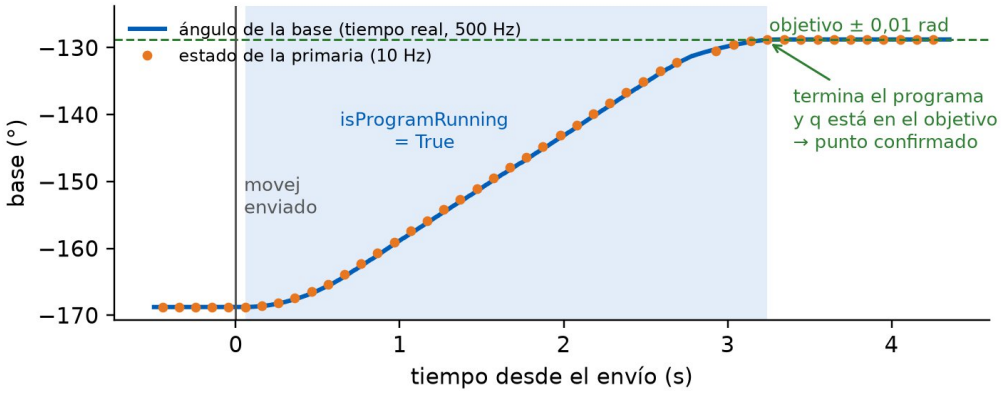


Fig. 7 · Datos reales de URSim. El robot no responde a un movej: el punto se da por bueno cuando el programa termina y el ángulo está a menos de 0,01 rad.

Además

- **Repositorio GitHub** (github.com/godzilin/Practica1UR): cada cambio es un commit; README con instalación y uso.
- **A prueba de fallos:** reconexión automática, mensajes corruptos descartados, configuración validada. Probado con 20 000 mensajes rotos, cortes de red y URSim reiniciado.
- **Seguridad:** `stopj` si el programa se cierra con el robot en marcha; velocidad ≤ 0,3 rad/s; una parada de emergencia nunca se desbloquea por red.

Robot real

[Completar tras la sesión: qué UR3e se usó y diferencias con URSim.]